



DevForge AI

AI-powered game development toolkit for the Unity Editor

Product Documentation

Version 0.9.0 · Unity 6 (6000.0+) · September 2026

Developed by GetSolveit



Contents

1. Introduction	4
1.1 Key principles	4
1.2 What you can do	4
1.3 Architecture	5
2. Installation & Requirements	5
2.1 Requirements	5
2.2 Install the package	6
2.3 Folder structure	6
3. Getting Started	6
3.1 Connect an AI provider	7
3.2 Project context and memory	9
4. How DevForge AI Works	9
4.1 Life of an AI request	9
4.2 How changes are applied	9
4.3 Build a game or feature from AI Chat	10
4.4 Code Agent workflow	10
5. Page Reference	11
5.1 Home	11
5.2 AI Chat	11
5.3 Code Assistant	12
5.4 Project Analyzer	14
5.5 Scene Assistant	14
5.6 GameObject Assistant	15
5.7 Prefab Assistant	16
5.8 Game Systems	17
5.9 Asset Tools	17
5.10 Shader & Material	18
5.11 Animation & Audio	19
5.12 UI/UX Tools	20
5.13 Optimization	20
5.14 Console Fixer	21
5.15 Build Tools	22
5.16 Testing & QA	23
5.17 Agent Orchestrator	23
5.18 Documentation	24
5.19 Product Guide (in the editor)	26



- 5.20 Settings 26
- 6. Safety, Privacy & Data 28**
 - 6.1 What leaves your computer 28
 - 6.2 What stays on your computer 28
 - 6.3 Protections 28
- 7. Troubleshooting 29**
- 8. Limitations & Roadmap 29**
 - 8.1 Known limitations 29
 - 8.2 Planned features 30
 - 8.3 Tips 30



1. Introduction

DevForge AI is an extension for the Unity Editor (Unity 6) that brings AI assistance, project analysis and safe automated fixes into one window. You describe what you want in plain language: a question about your code, a bug to fix, a scene to set up, or a complete feature. DevForge AI collects the relevant project context, asks the AI tool you already use, and shows every proposed change for your review before anything in the project is modified.

1.1 Key principles

Principle	What it means for you
Your AI, your login	DevForge AI has no AI service, account or subscription of its own and needs no API keys . It runs the official tool of your provider (Claude Code, OpenAI Codex CLI, Google Antigravity CLI, Cursor CLI or a local model server) with your own login.
AI is read-only	The AI tools run in their read-only modes. They can read project files (if you allow it) but cannot write files or run commands. Only DevForge AI writes, and only what you approved.
Review before apply	Code changes are shown as diffs; scene/asset changes as operation or fix lists. Nothing is applied until you confirm.
Undo & rollback	Scene edits are one Undo step (Ctrl+Z). Files, assets and settings are backed up and can be rolled back from History.
Local & private	No telemetry and no DevForge server. Chat history, backups and reports stay in your project's Library folder.
Only your project	Counts, analyzers, fixers and generated documentation cover only your own files. DevForge AI's own folder and its log messages are never counted or changed.
Verified results	After changes, Unity compiles and DevForge AI reports compile errors, Console errors and test results honestly.

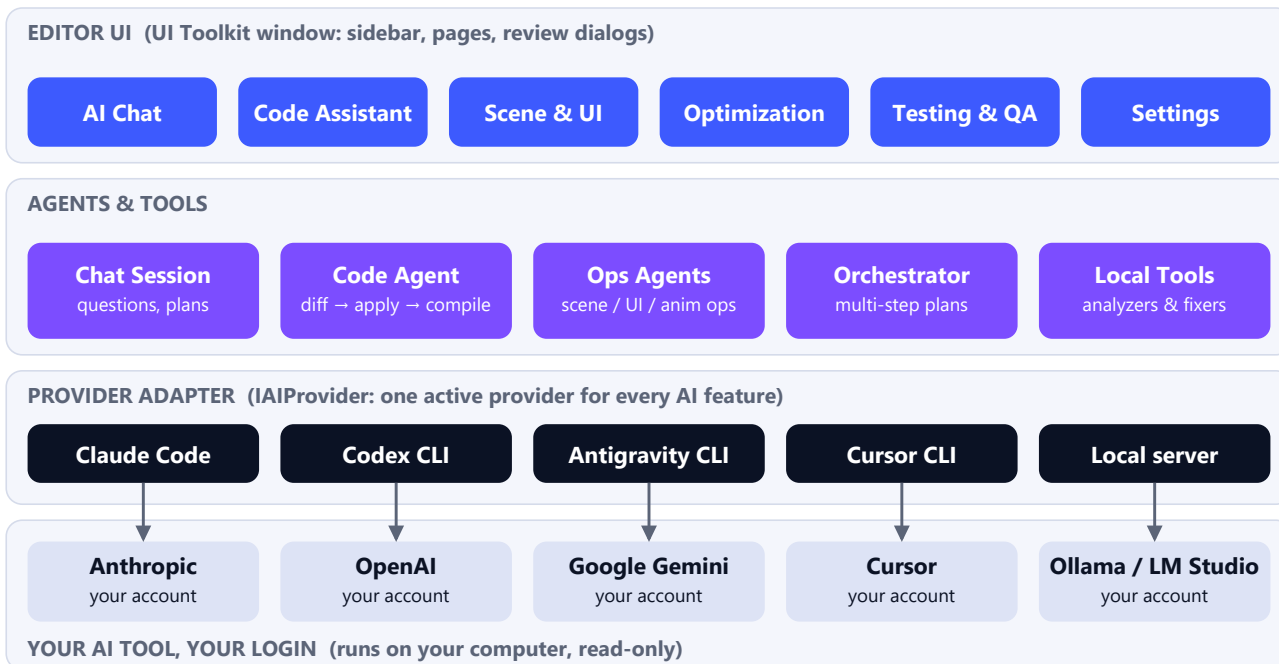
1.2 What you can do

- Chat with AI about your project with automatic context (project, scene, selection, console, project memory).
- Build a game or feature from one description: the AI creates a development plan, and after approval the agents create scripts, GameObjects, components, references, prefabs and UI step by step, then compile and verify.
- Write, fix and refactor C# code with the Code Agent (plan → diff → apply → compile check → AI error fixing).
- Create and edit scene objects, prefabs, materials, UI and Animator Controllers from text, with Undo support.
- Analyze and fix project problems locally: missing scripts, lighting, textures, audio, models, materials, prefabs, UI, platform and build settings.
- Optimize for performance, mobile, WebGL, graphics, memory and build size.
- Run tests, smoke tests and a one-click **Verify Project** before builds.

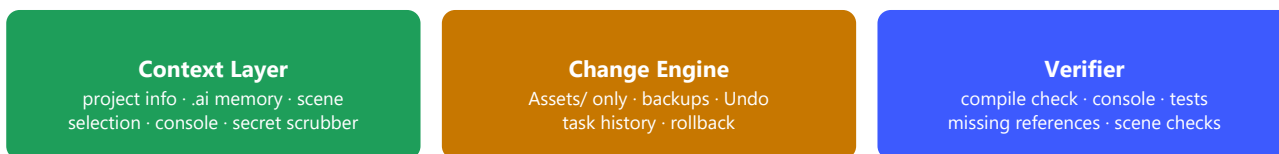


1.3 Architecture

DevForge AI is organised in layers. The editor UI talks to agents and local tools; every AI request goes through a single provider adapter to the one provider you marked as active; changes go through the Change Engine; results are checked by the Verifier.



Layered architecture: UI → agents & tools → provider adapter → your own AI tool.



Shared services used by every agent and tool.

2. Installation & Requirements

2.1 Requirements

Area	Supported
Unity	6000.0 LTS and newer (developed on 6000.0.58f2).
Editor OS	Windows (verified). macOS and Linux are supported by the code but not yet tested.
AI tool (one of)	Claude Code CLI 2.1+ · OpenAI Codex CLI · Google Antigravity CLI (<i>agy</i>) · Cursor CLI · a local OpenAI-compatible server (Ollama, LM Studio).
Render pipeline	URP (full), Built-in (most tools), HDRP (analysis only).
Required packages	None. Unity UI (ugui), Test Framework, TextMeshPro, Shader Graph and URP are optional; tools that need them switch themselves off when missing.



2.2 Install the package

- In Unity: **Assets > Import Package > Custom Package...** and select *DevForgeAI_0.9.0.unitypackage*. Import everything.
- The plugin lives in *Assets/DevForgeAI* (it can be moved to any folder under Assets).
- Open it with **Tools > DevForge AI > Open DevForge AI** (also under **Window > DevForge AI**).
- On first start the **Getting Started** page guides you through setup.

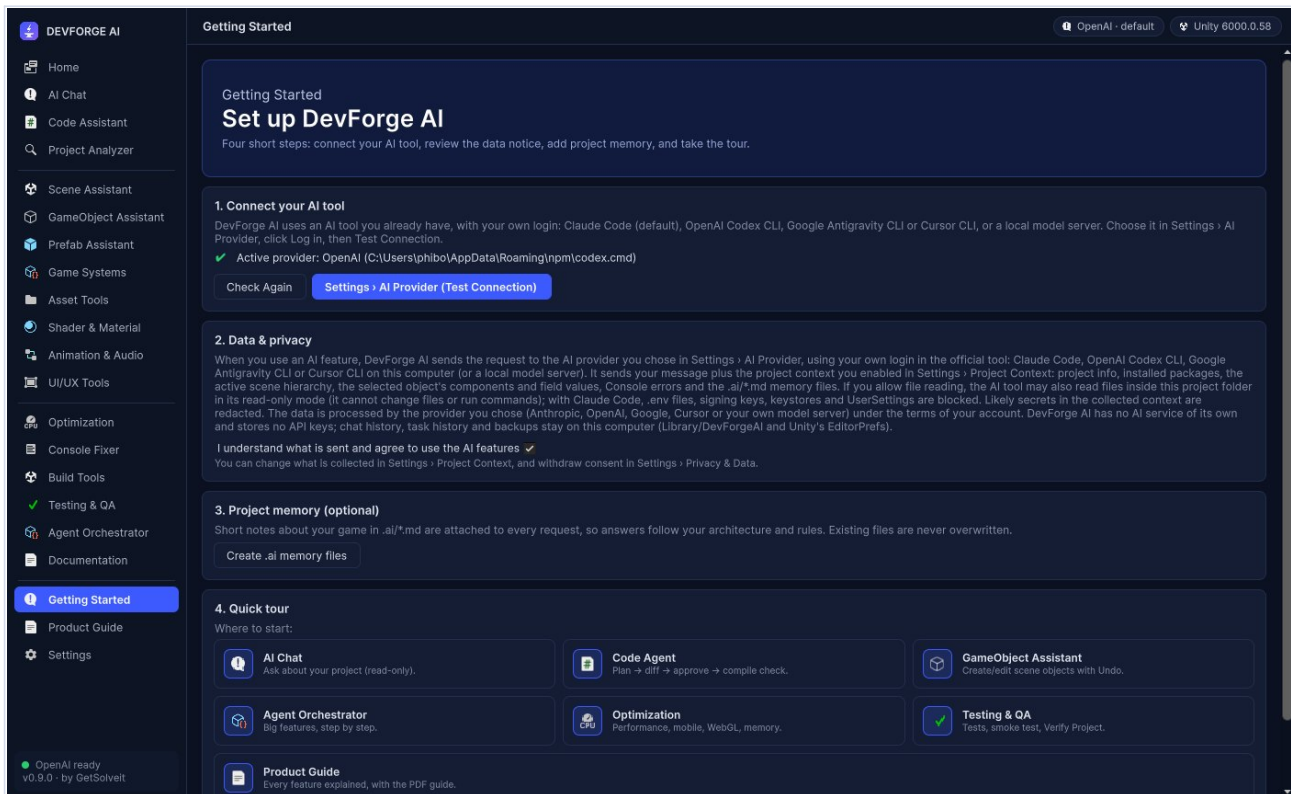
2.3 Folder structure

Folder / file	Contents
Editor/	All editor code (assembly <i>DevForgeAI.Editor</i> , editor-only, never included in builds).
Editor/UI/Styles, Editor/UI/Icons	Stylesheet (DevForge.uss) and the DevForge icon.
Integrations/TestRunner	Optional bridge to the Unity Test Framework (compiled only when the package exists).
Tests/	EditMode tests of DevForge AI itself.
Documentation/	User Guide, AI Providers, Privacy, Security, Compatibility, Known Limitations and this PDF.
Library/DevForgeAI/ (per project)	Chat history, agent state, backups, reports. Not version-controlled.
.ai/*.md (optional, per project)	Your project memory: short notes sent with every AI request.

3. Getting Started

The Getting Started page runs on first use (and any time from **Tools > DevForge AI > Getting Started**). It has four steps:

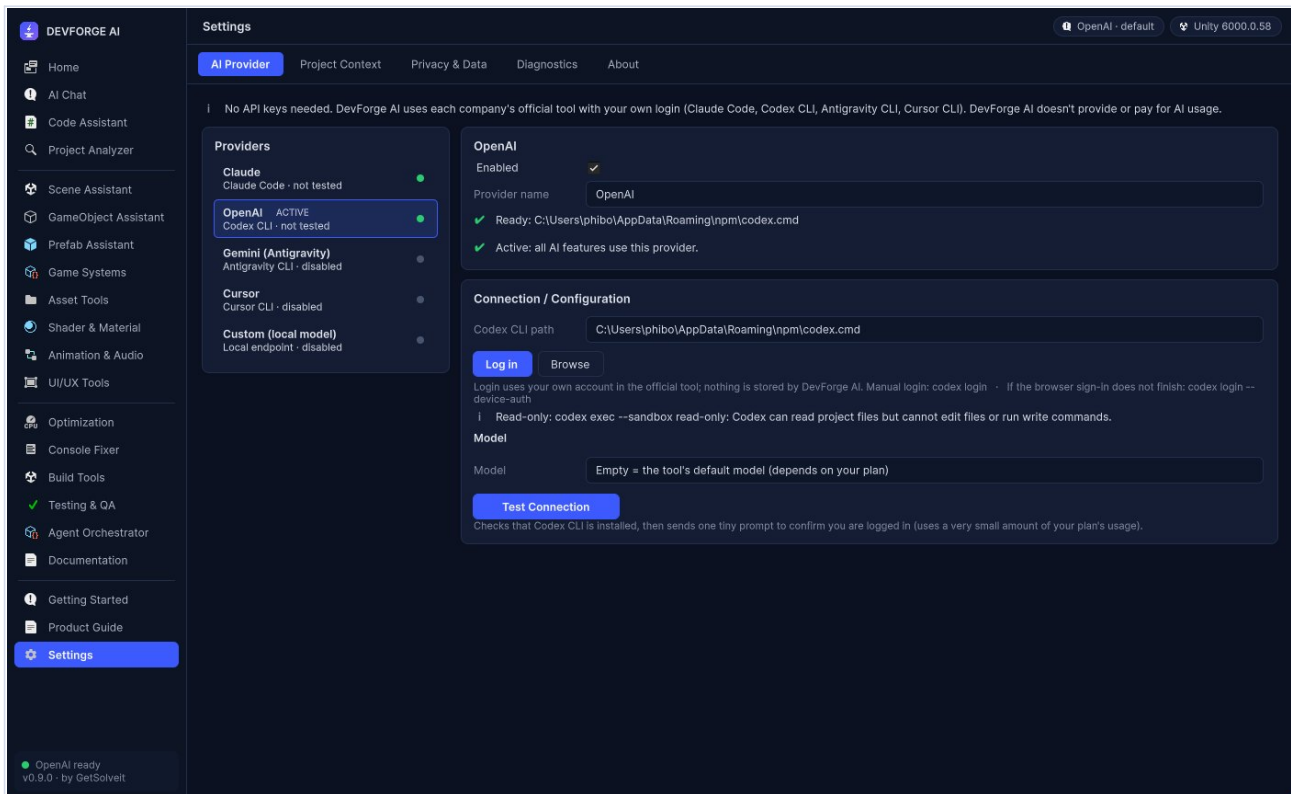
- **1. Connect your AI tool** – shows the active provider and whether it is ready.
- **2. Data & privacy** – explains exactly what is sent. AI features stay off until you accept; local tools always work.
- **3. Project memory** – creates *.ai/project.md*, *architecture.md*, *coding-rules.md*, *systems.md*, *decisions.md* (existing files are never overwritten).
- **4. Quick tour** – shortcuts to the most important pages, including the in-editor **Product Guide**.



Getting Started: provider status, data notice and consent, project memory and quick tour.

3.1 Connect an AI provider

Open **Settings** > **AI Provider**. Each provider has *Enabled*, a display name, the tool path (detected automatically), a **Log in** button, an optional model, **Test Connection** and its status. The provider marked **ACTIVE** is used by every AI feature, and its name appears in the header chip, in chat replies and in progress messages.



Settings > AI Provider: provider list with status (left) and configuration of the selected provider (right).

Provider	Official tool	Install	Log in
Claude (default)	Claude Code CLI	claude.com/claude-code	Log in button or <i>claude auth login</i>
OpenAI	Codex CLI	<i>npm install -g @openai/codex</i>	<i>codex login</i> (ChatGPT account); fallback <i>codex login --device-auth</i>
Gemini	Antigravity CLI (<i>agy</i>)	PowerShell: <i>irm https://antigravity.google/cli/install.ps1 iex</i>	run <i>agy</i> once (Google sign-in)
Cursor	Cursor CLI (<i>agent</i>)	cursor.com/cli	<i>agent login</i>
Custom	Your local server	e.g. Ollama, LM Studio	none (optional token from an environment variable)

Steps for every provider

- Install the tool, then select the provider in Settings > AI Provider and tick **Enabled**.
- Click **Log in**. DevForge AI first asks the tool whether you are signed in: if you are, it simply says so (a terminal can still be opened to switch accounts); otherwise a terminal opens with the tool's own sign-in, which you finish in the browser.
- Optional: enter a **Model** (empty = the tool's default for your plan).
- Click **Test Connection** (version check, login check, one tiny prompt).
- Click **Use for AI features** to make it the active provider.

Gemini note: Google retired Gemini CLI for individual accounts on 18 June 2026. DevForge AI's Gemini provider uses the **Antigravity CLI** (*agy*), never the Antigravity IDE app: a path to the old Gemini CLI or to Antigravity.exe is replaced automatically, and Browse refuses them. Leave the path empty to auto-detect %LOCALAPPDATA%\agy\bin\agy.exe. Antigravity is told not to run commands and to read only a few files; if it still tries a blocked action, DevForge AI asks once more automatically. Answers can take 20–60 seconds. If you already



use the Antigravity IDE with the same Google account, the CLI may already be signed in (check with *agy models*).

How each tool is kept read-only

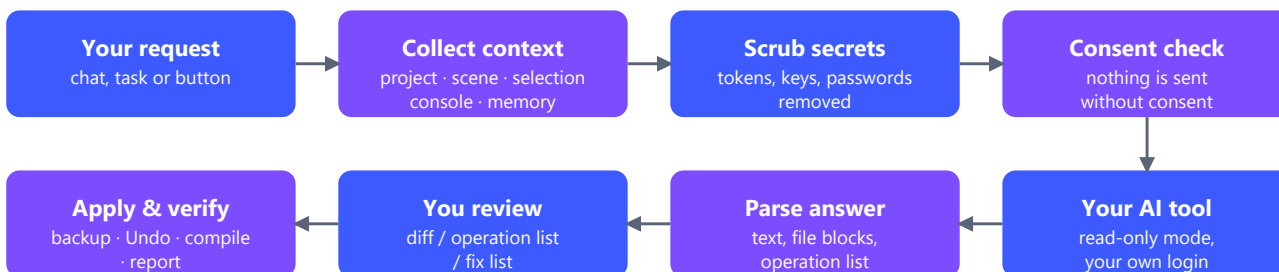
Provider	Command used by DevForge AI	Read-only guard
Claude	<code>claude -p --output-format stream-json --tools Read,Grep,Glob ...</code>	Only read tools; deny rules for .env, keys, keystores; project hooks/MCP not loaded by default.
OpenAI	<code>codex exec --sandbox read-only --ephemeral -</code>	Codex read-only sandbox; prompt on standard input.
Gemini	<code>agy --output-format json -p "Read the file ..."</code>	Headless mode auto-denies writes and commands; <code>--dangerously-skip-permissions</code> is never used.
Cursor	<code>agent -p --mode ask --output-format text</code>	Ask mode; <code>--force</code> is never used.
Custom	HTTP (OpenAI-compatible) to your server	No tools at all; HTTPS required except localhost.

3.2 Project context and memory

Settings > Project Context controls what is attached to AI requests: project info, *.ai* memory files, the active scene hierarchy, the selected object, Console errors, and whether the AI may read project files. Turn off what you don't need to save usage. Likely secrets (API keys, tokens, private keys, *password=* values) are removed before sending.

4. How DevForge AI Works

4.1 Life of an AI request



Every AI feature follows the same path; nothing changes in the project before step 7.

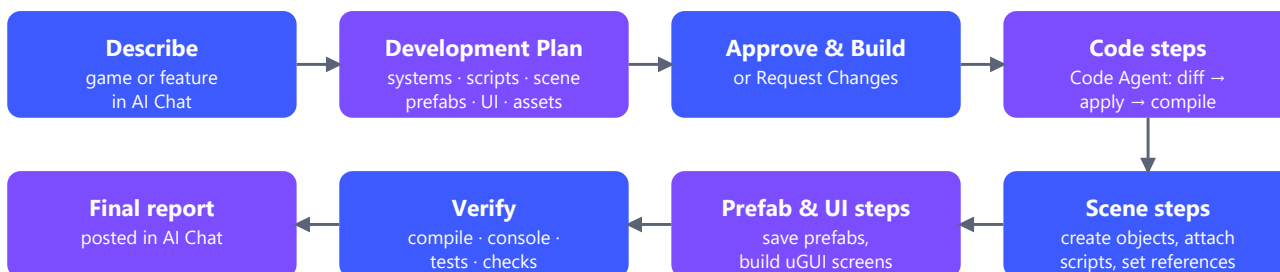
4.2 How changes are applied

Kind of change	How you approve	How to undo
Code (Code Agent)	Per-file diff with checkboxes and a confirmation dialog	Rollback in Code Assistant > History
Scene objects (agents, scene fixers)	Operation / fix list + confirm	Ctrl+Z (one Undo step). Scenes are never saved automatically.
Assets & import settings	Fix list + confirm	Rollback: asset and .meta are backed up
Project settings	Fix list + confirm	Rollback restores the recorded old values
Created assets (materials, prefabs, controllers)	Part of the list	Rollback moves them to the system Trash

Risky fixes are unchecked by default. Deletions always need explicit selection and go to the Trash. Files are only written inside *Assets/*, and the DevForge AI folder itself is protected.

4.3 Build a game or feature from AI Chat

When you ask AI Chat to *build, create, implement or add* something (for example "Create a 3D 3-lane runner: move left/right, collect coins, avoid obstacles, score and game over UI"), the AI does not just print scripts. It inspects the project (including a list of existing scripts, prefabs and scenes so it can reuse them) and returns a **Development Plan** with the sections Game/Feature, Systems, Scripts, GameObjects & Components, References, Prefabs, UI, Assets and Verification, plus an ordered list of build steps. If important details are missing it asks at most three short questions first.



Build workflow: each step is reviewed by you before it is applied.

After approval the plan opens in the **Agent Orchestrator**. Steps run in order: first all scripts (so Unity can compile them), then scene steps that create GameObjects, attach the new scripts, configure components and assign serialized references, then prefabs and UI, and finally verification. If Unity reports compile errors, **Ask AI to Fix Errors** sends them back to the AI (up to 3 times); a failed verification offers **Fix Errors with Code Agent**. Models, textures and audio that cannot be generated are replaced by primitives and listed as *Manual asset required*.

Final report

IMPLEMENTED

- Scripts: PlayerController.cs, CoinSpawner.cs, ...
- GameObjects: Create GameObject 'Player' (Capsule), ...
- Components: Add PlayerController to \$player, Set ...
- Prefabs: Save \$coin as prefab Assets/Prefabs/Coin.prefab
- Scenes: Assets/Scenes/Main.unity (modified, not saved yet: press Ctrl+S)
- UI: Create UI Text 'ScoreText', ...
- Assets: Create material Assets/Materials/Coin.mat

CHANGED FILES

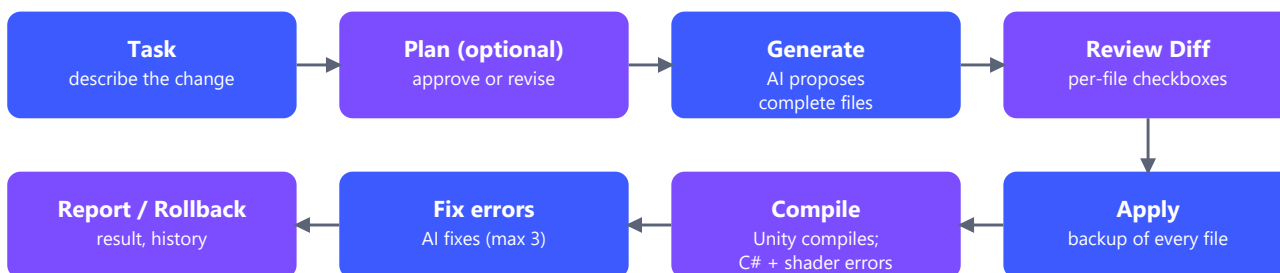
- Assets/Scripts/PlayerController.cs ...

VERIFICATION

- Compile: OK
- Console: 0 error(s), 0 warning(s)
- Tests/validation: ...
- Remaining issues: none

The report is built from what the agents actually did (applied files and operations, real compile and Console state). The example lines show the format only.

4.4 Code Agent workflow



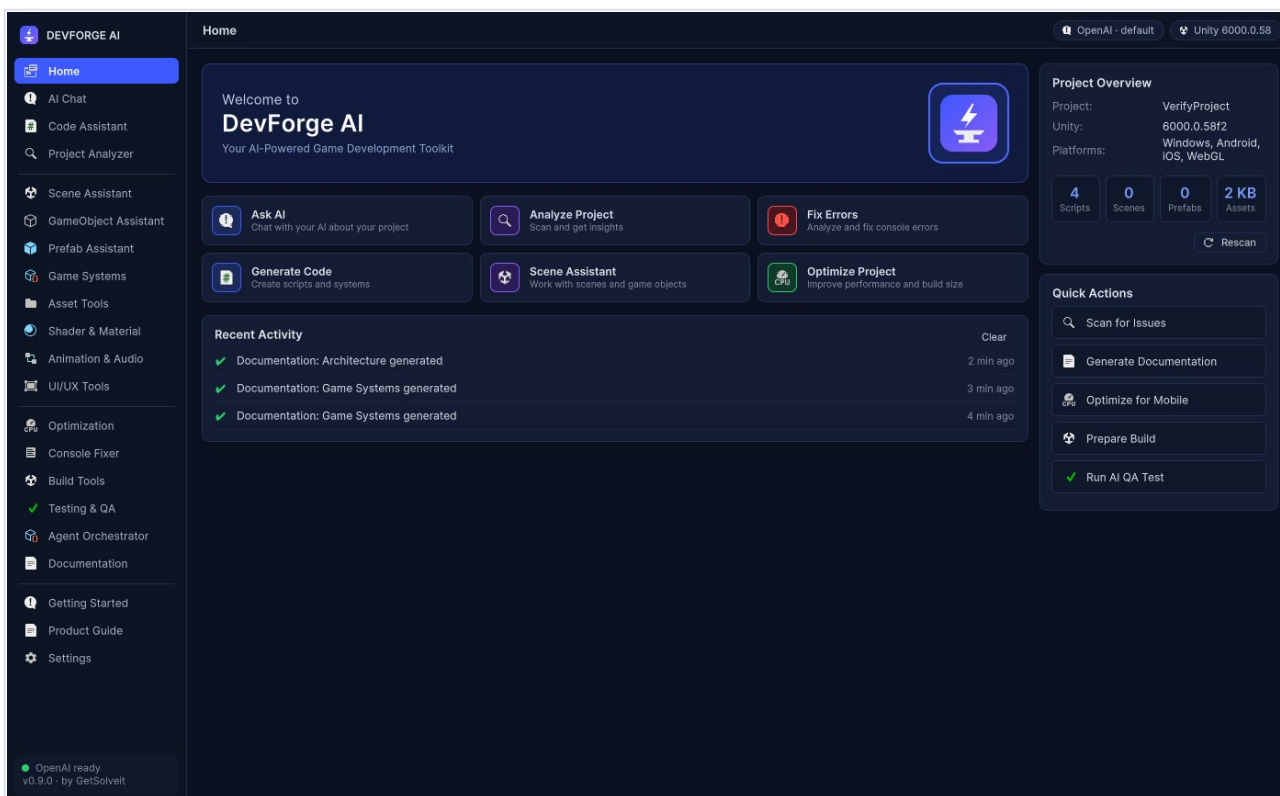
Code Agent: the AI only proposes; DevForge AI writes the approved files and verifies them.

5. Page Reference

The sidebar groups the pages by purpose. Every page below is shown as it appears in the Unity Editor.

5.1 Home

Welcome screen with shortcuts to the main tools, a **Project Overview** (project, Unity version, target platforms, and counts of *your* scripts, scenes, prefabs and asset size; DevForge AI's own files are not counted) with **Rescan**, **Quick Actions** (Scan for Issues, Generate Documentation, Optimize for Mobile, Prepare Build, Run AI QA Test) and **Recent Activity**.



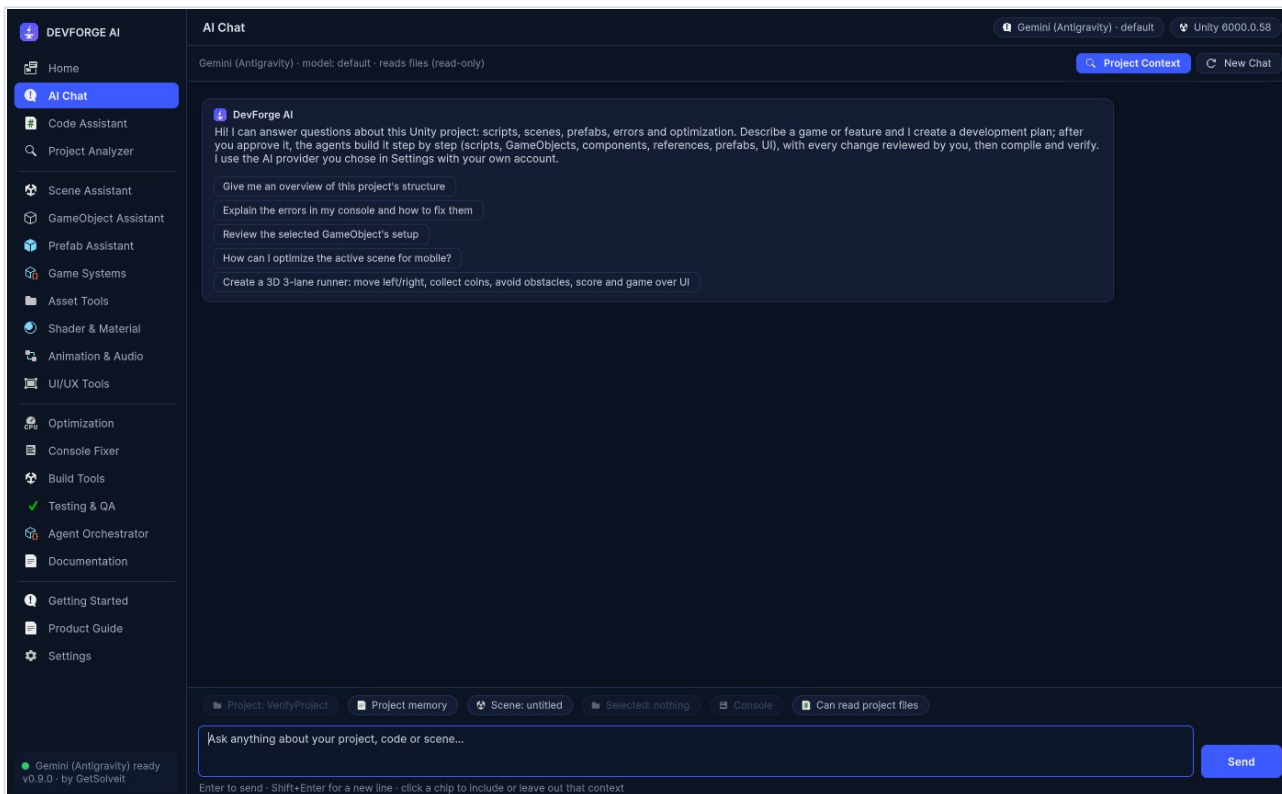
Home page.

5.2 AI Chat

Ask anything about the project. The **context chips** above the input show what is sent with your message (project info, project memory, scene, selected object, Console, file reading): **click a chip** to include or leave out that context; switched-off chips stay visible, dimmed, and a tooltip explains each one. **Project Context** turns all context off or on. Replies show the provider's name, code appears in blocks with a Copy button, and build requests return a Development Plan with **Approve & Build**. **Enter** sends, **Shift+Enter** adds a



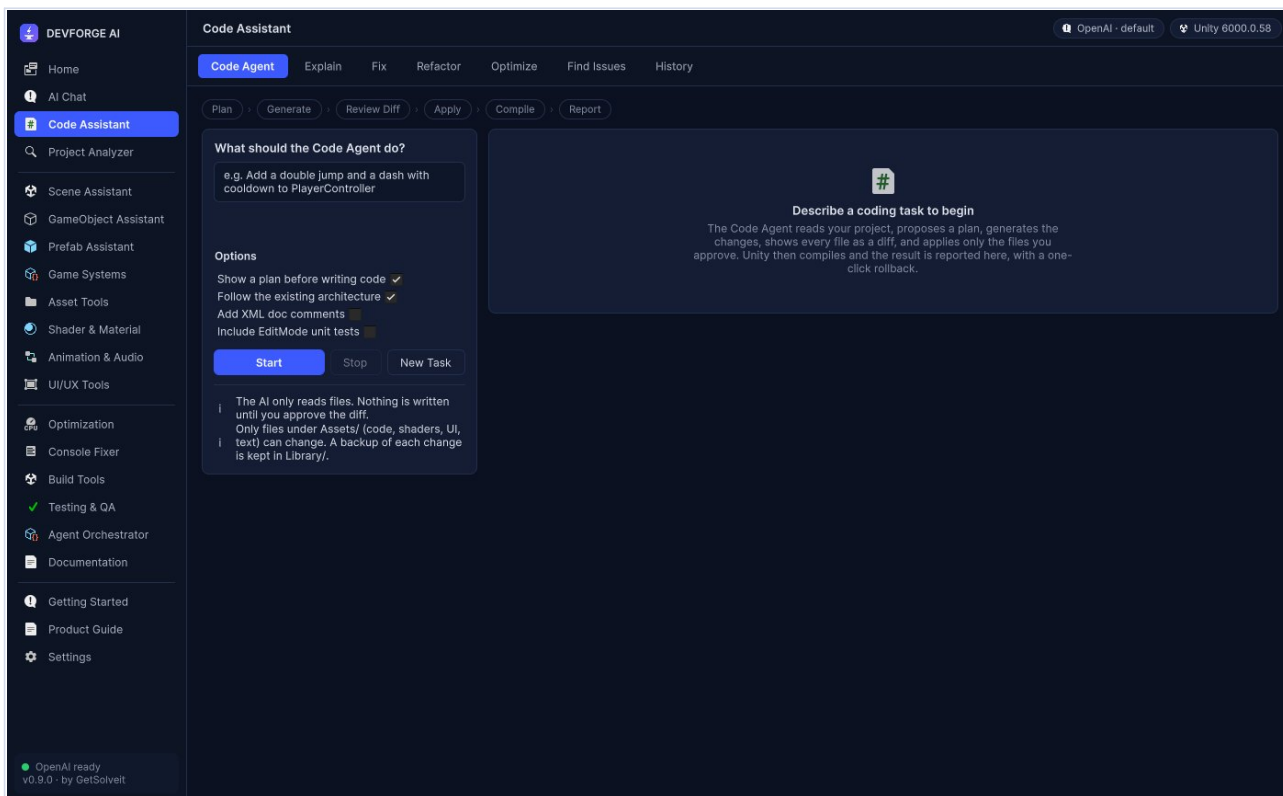
line, **New Chat** starts over. History is kept per project.



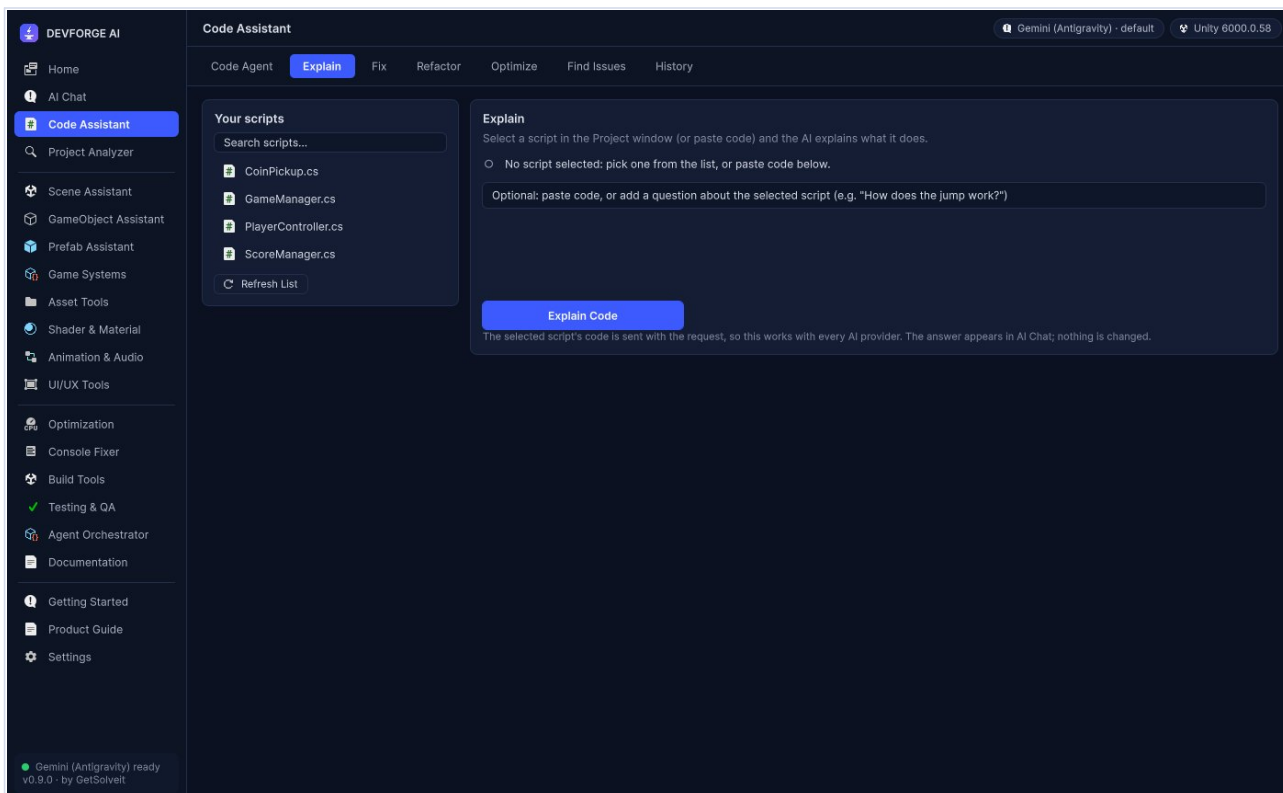
AI Chat page.

5.3 Code Assistant

Tabs: **Code Agent** (plan → generate → review diff → apply → compile → report, with options for a plan first, following the existing architecture, XML comments and EditMode tests), **Explain**, **Fix**, **Refactor**, **Optimize**, **Find Issues** and **History** (every applied task, asset fix and settings change, with rollback). **Explain**, **Optimize** and **Find Issues** list all of your project's scripts: pick one (optionally add a question) and the AI answers in AI Chat. The script's code is sent with the request, so this works with every provider, including local models.



Code Assistant page.

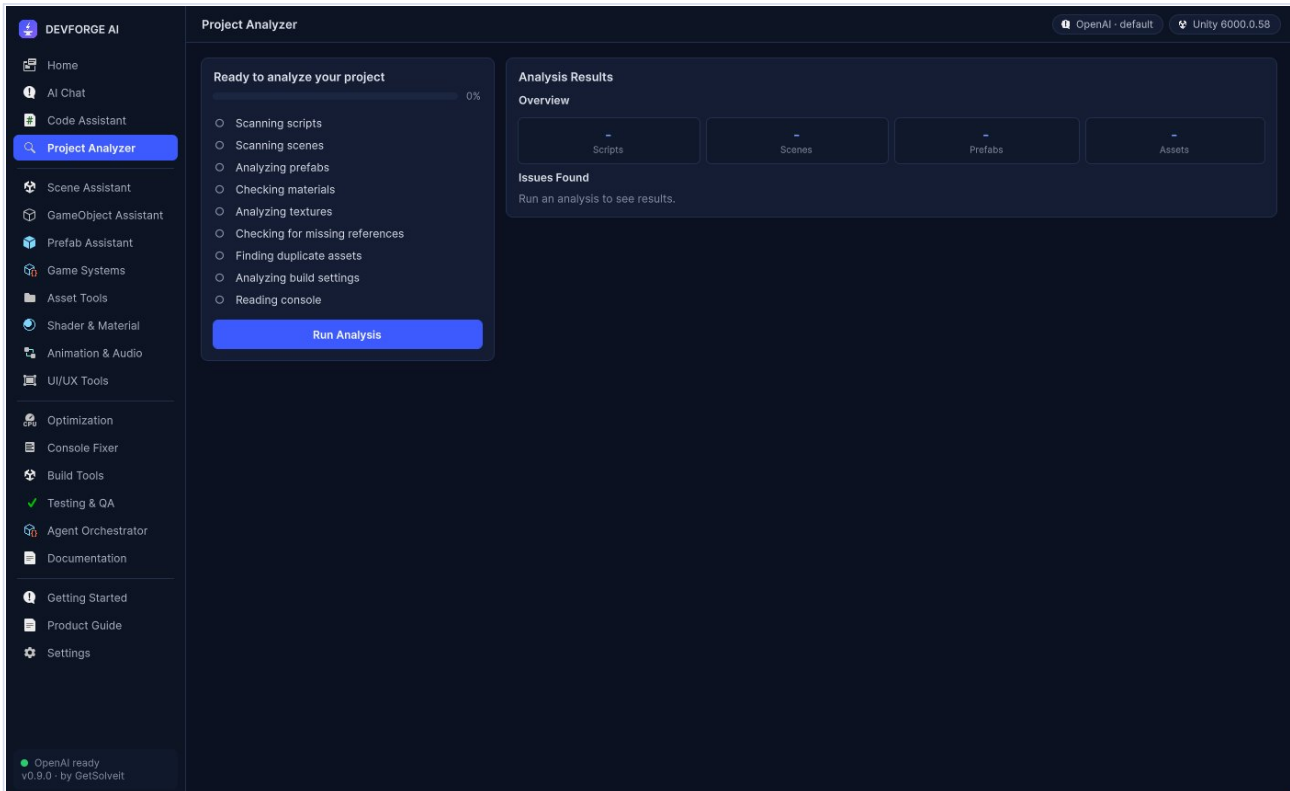


Code Assistant › Explain: pick any of your scripts (example project).



5.4 Project Analyzer

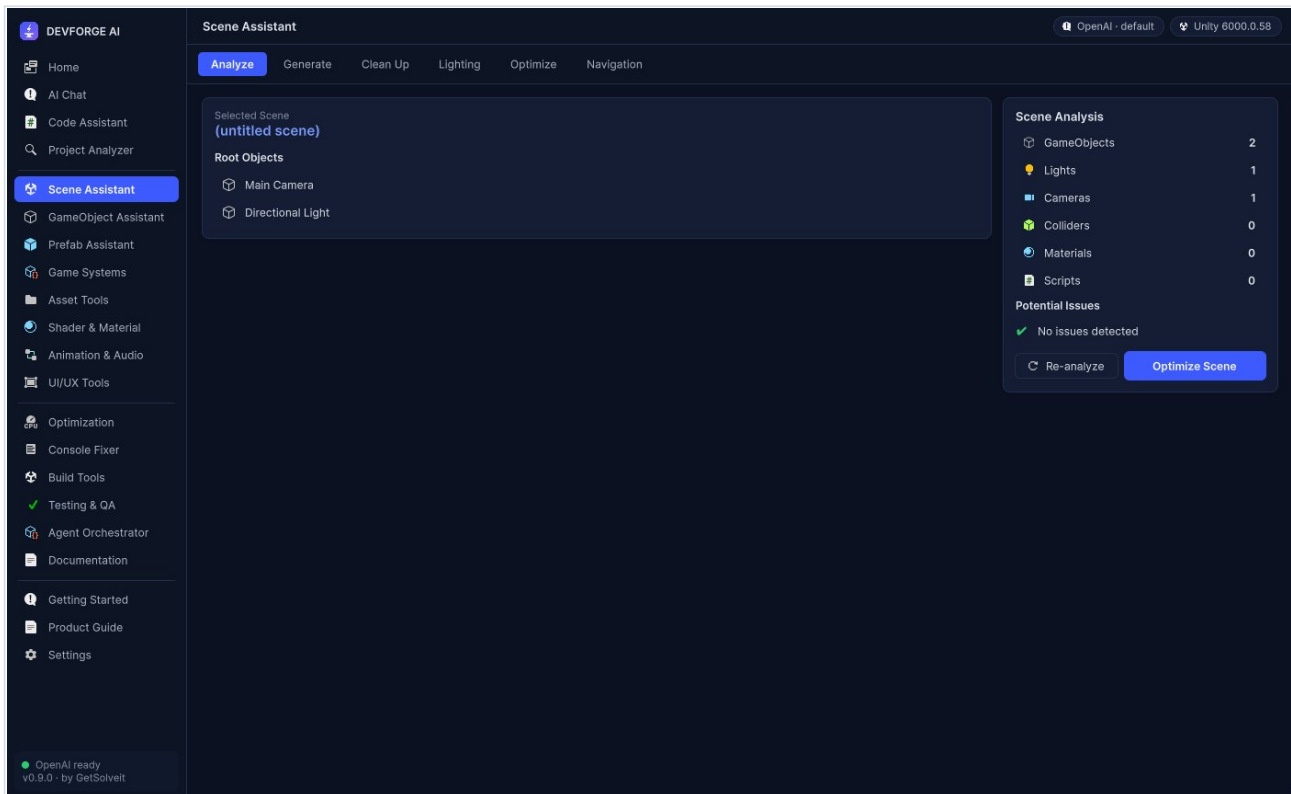
Local, rule-based scan of the whole project: scripts, scenes, prefabs, materials, textures, missing references, duplicate assets, build settings and the Console. Results are grouped with counts and issues.



Project Analyzer page.

5.5 Scene Assistant

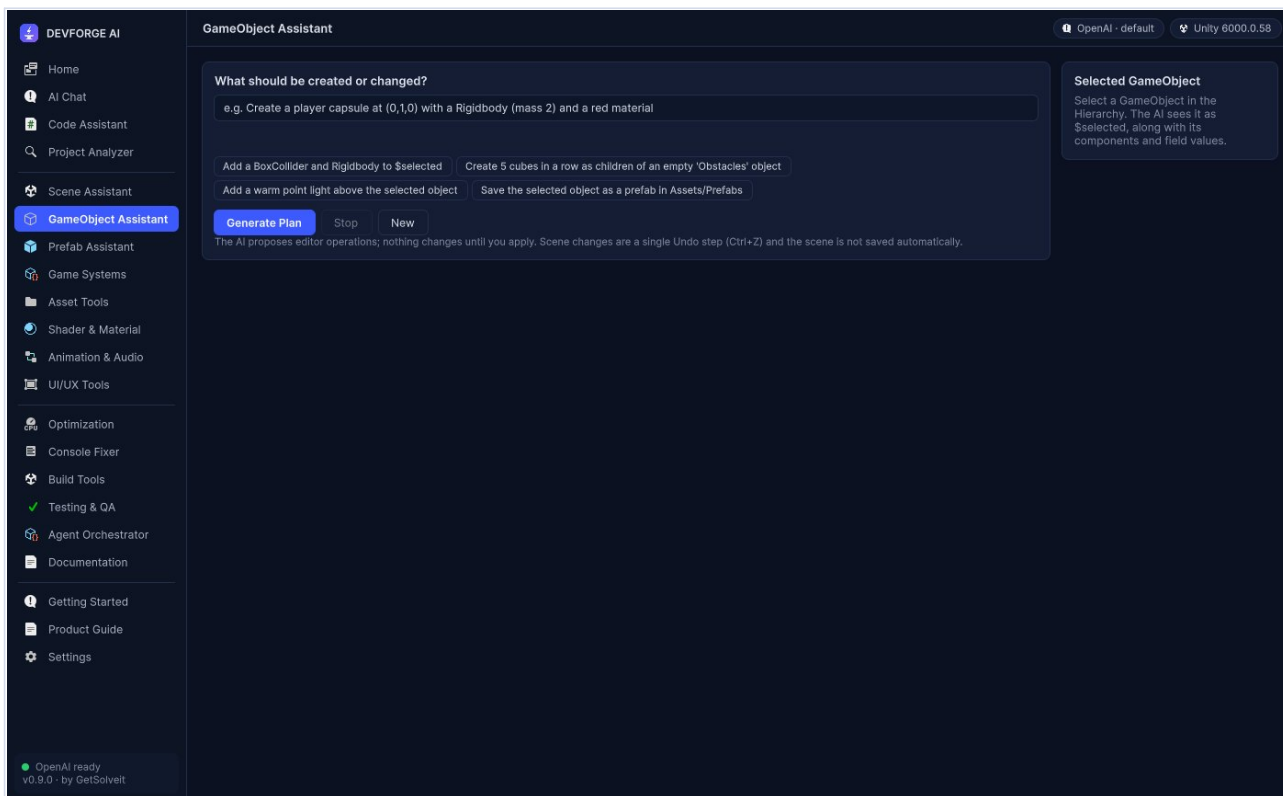
Analyze (root objects, counts of GameObjects, lights, cameras, colliders, materials, scripts, potential issues, **Optimize Scene** with AI), **Generate** (AI scene operations), **Clean Up** (missing scripts, empty objects, extra AudioListeners, missing EventSystem), **Lighting** and **Optimize**. *Navigation* is planned.



Scene Assistant page.

5.6 GameObject Assistant

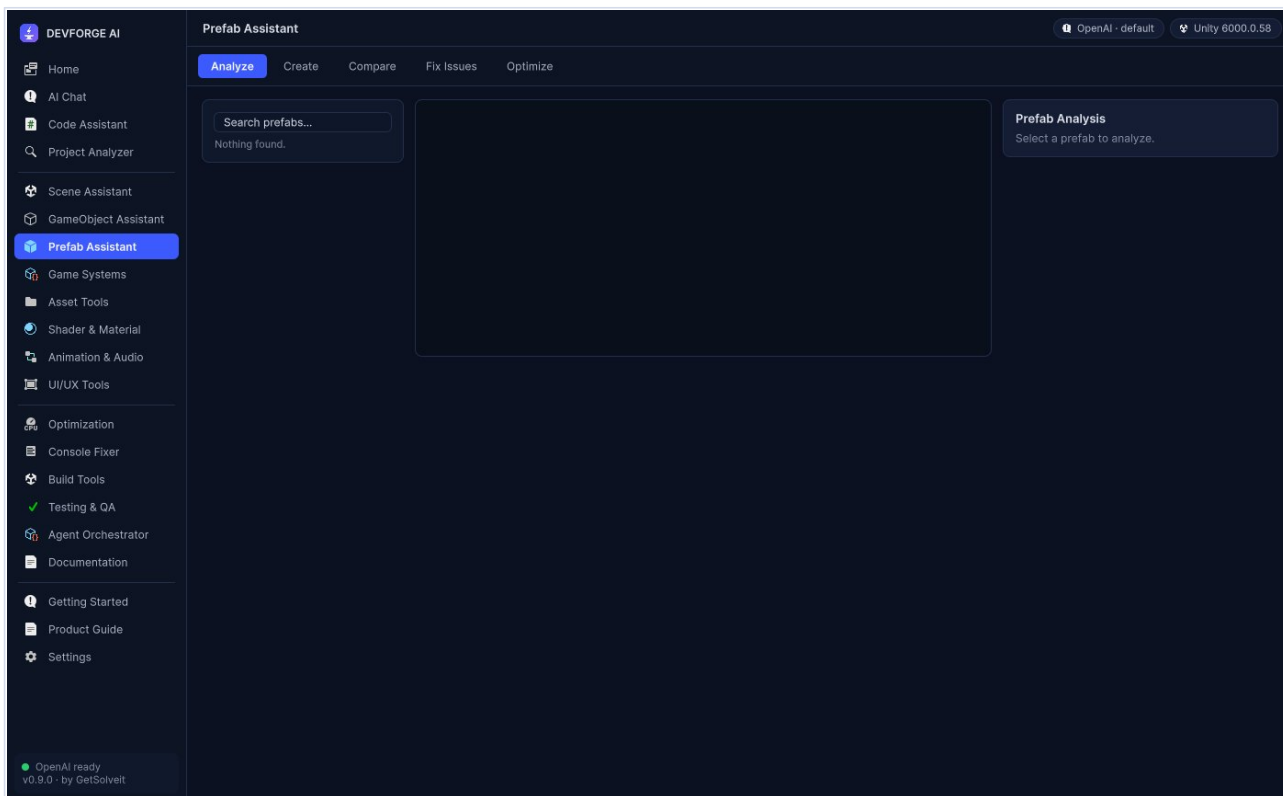
Describe objects, components, materials or prefabs to create or change (for example "*Create a player capsule at (0,1,0) with a Rigidbody (mass 2) and a red material*"). The AI returns an operation list that you review; *\$selected* refers to the selected object. All changes are one Undo step.



GameObject Assistant page.

5.7 Prefab Assistant

Analyze a prefab, **Create** prefabs from the selection, **Compare** two prefabs, **Fix Issues** (missing scripts) and **Optimize**.

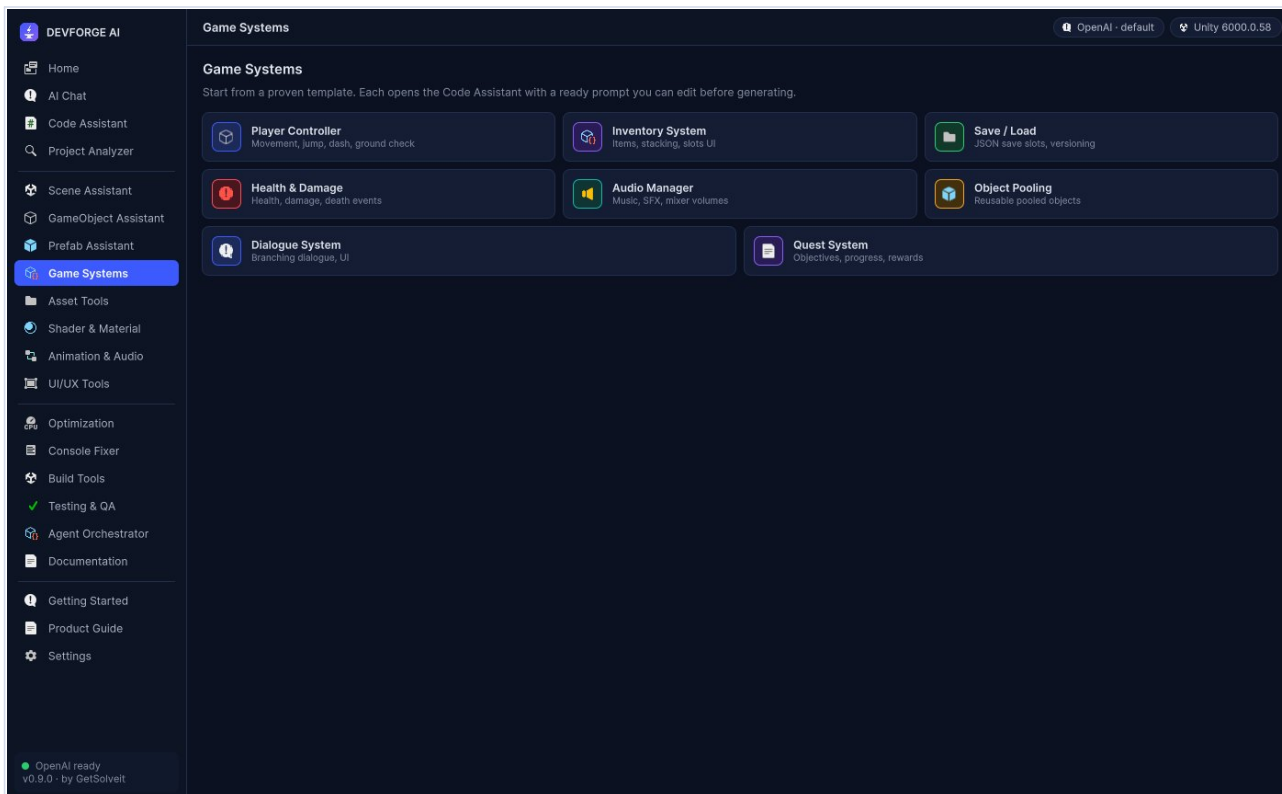


Prefab Assistant page.



5.8 Game Systems

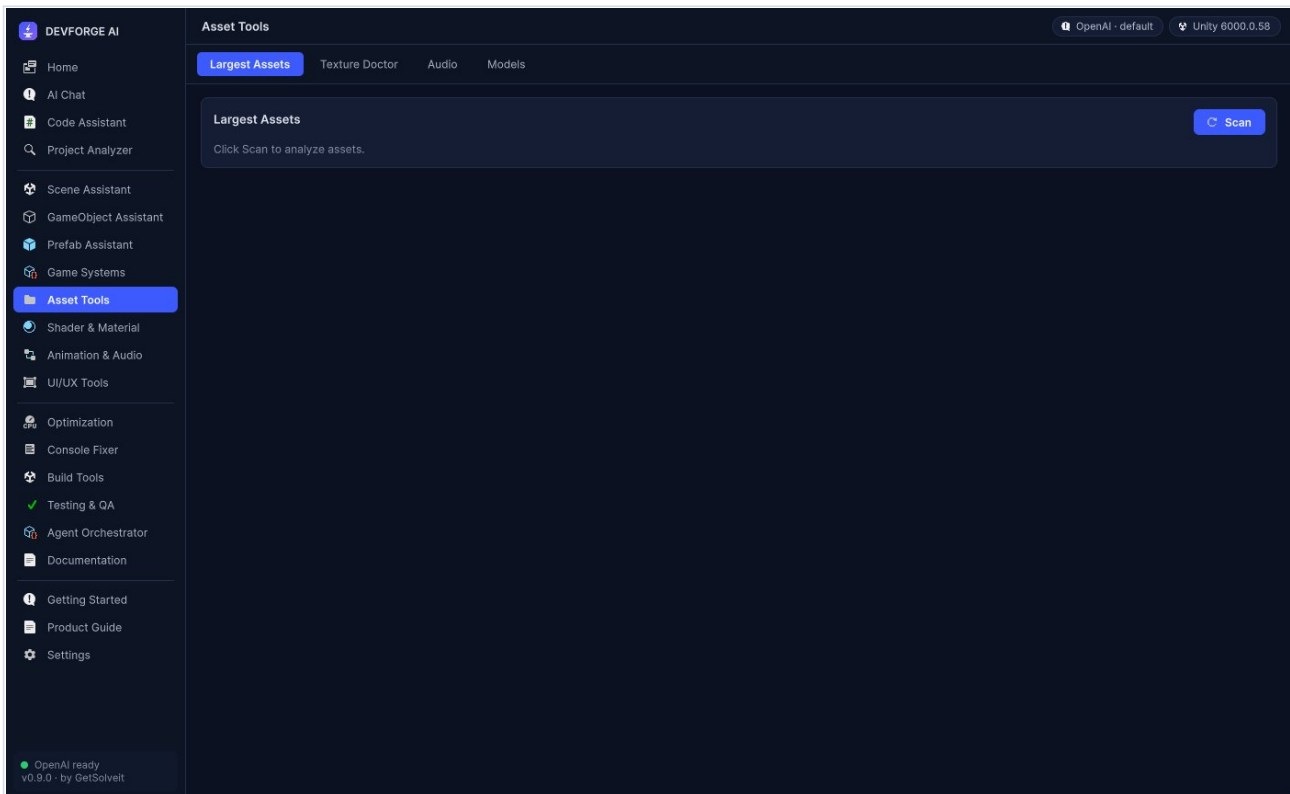
Proven starting points: Player Controller, Inventory, Save/Load, Health & Damage, Audio Manager, Object Pooling, Dialogue and Quest systems. Each opens the Code Agent with a ready prompt that you can edit.



Game Systems page.

5.9 Asset Tools

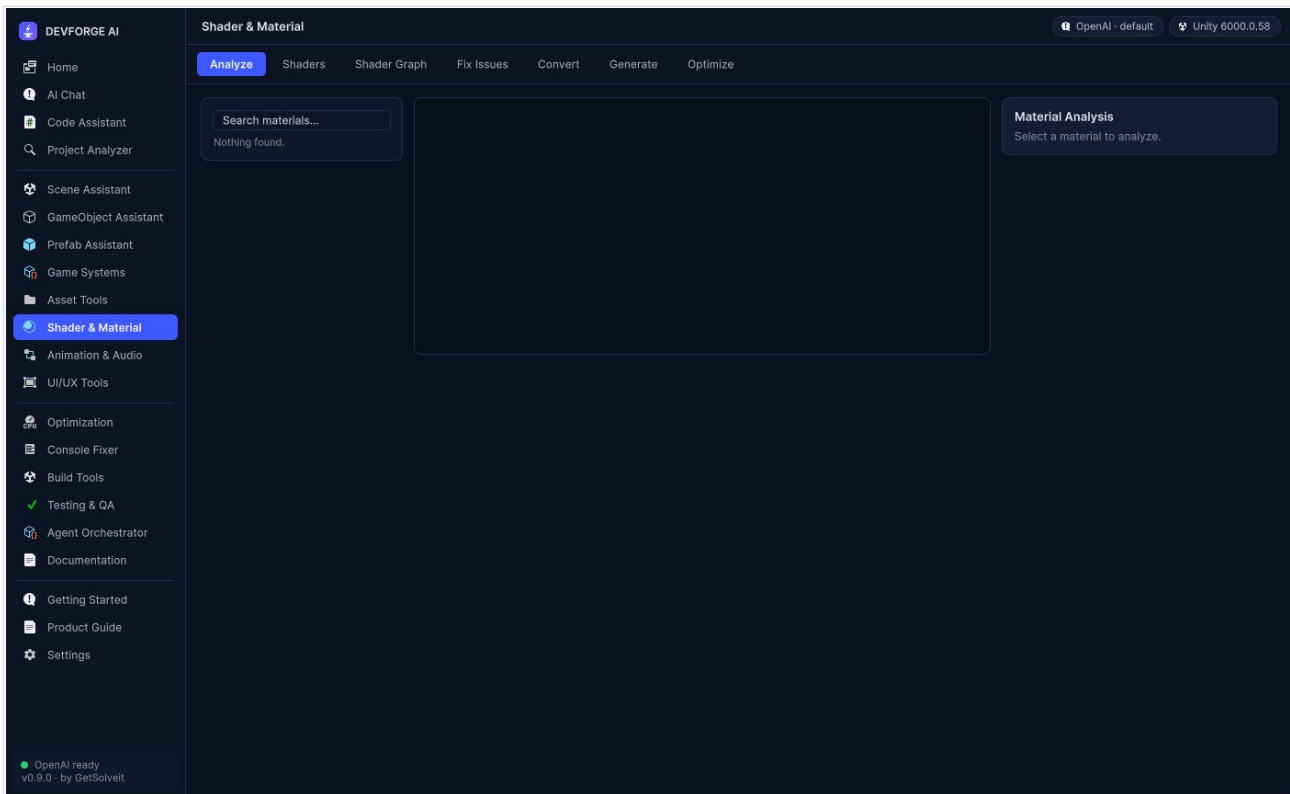
Largest Assets, **Texture Doctor** (import settings, compression, sizes), **Audio** and **Models** import fixes. Asset and .meta files are backed up before changes.



Asset Tools page.

5.10 Shader & Material

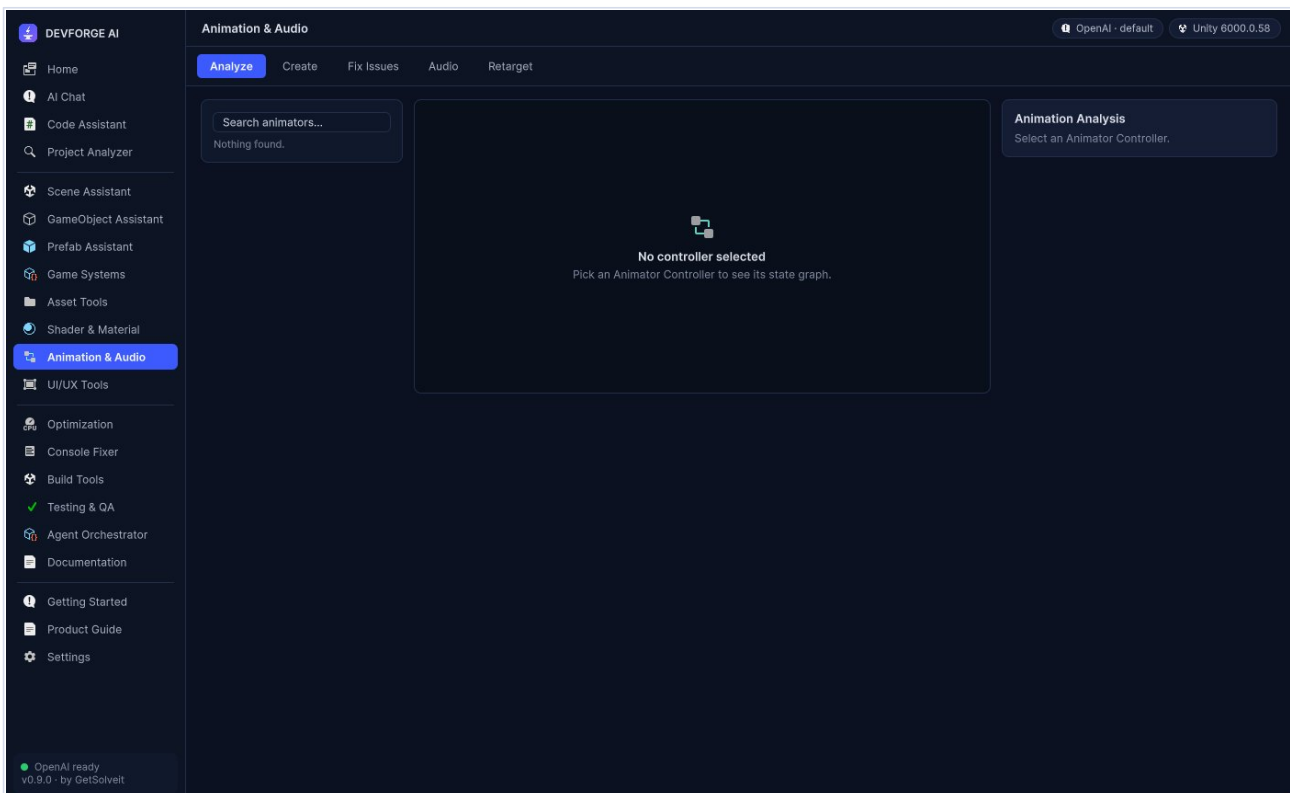
Material analysis; shader analysis (errors, URP compatibility); Shader Graph analysis; **Fix Issues**; Built-in → URP **Convert**; **Generate** new shaders or Custom Function HLSL via the Code Agent; **Optimize** (duplicate materials).



Shader & Material page.

5.11 Animation & Audio

Animator analysis with a state graph, **Create** Animator Controllers (quickly from clips, or with AI including blend trees and transitions), **Fix Issues** and **Audio** (AudioSource fixes). *Retarget* is planned.

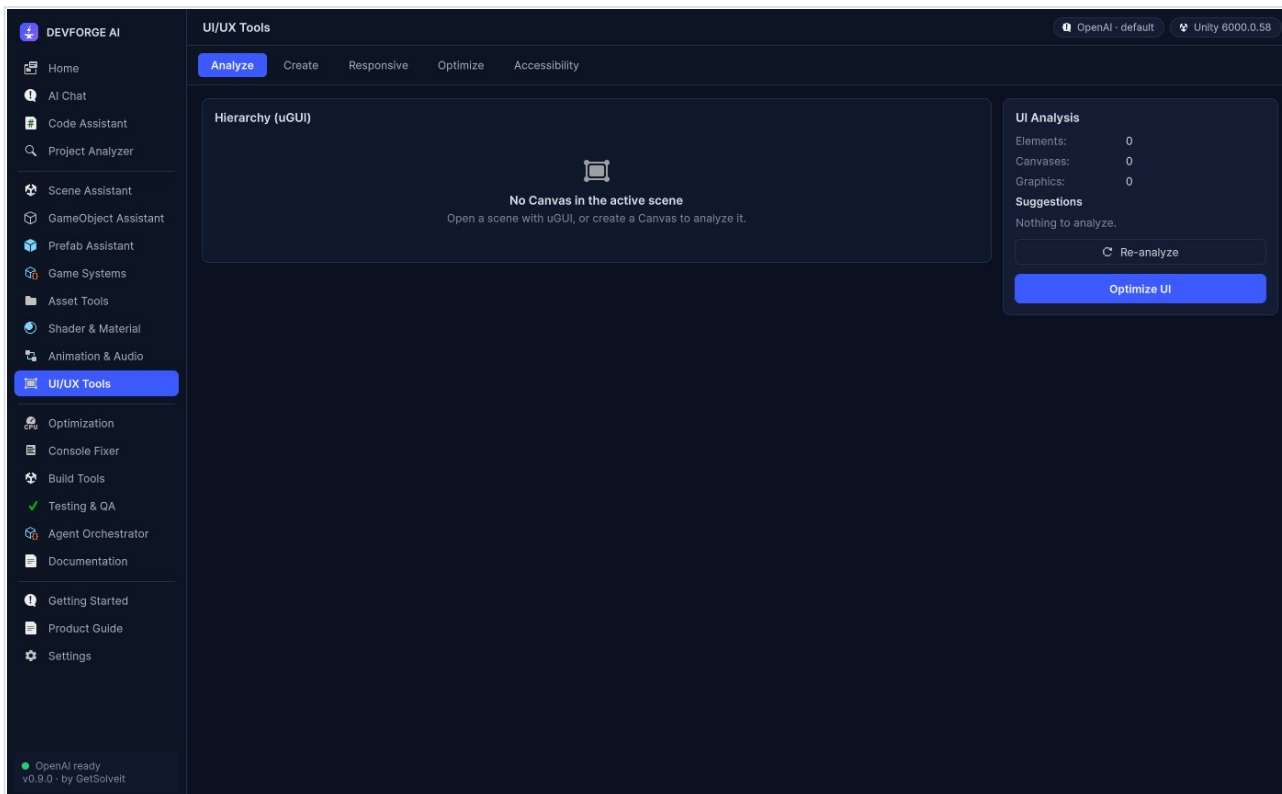


Animation & Audio page.



5.12 UI/UX Tools

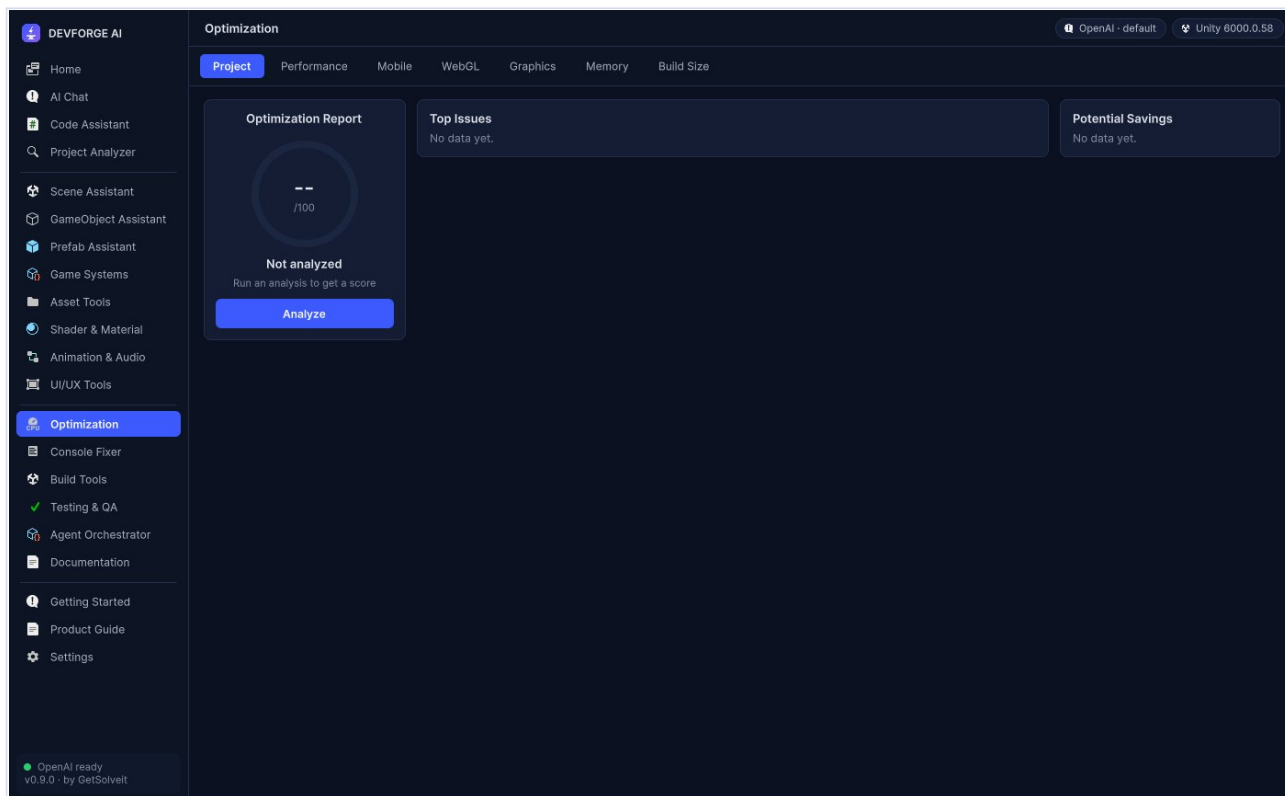
uGUI hierarchy analysis, AI UI creation (canvases, panels, buttons, text, sliders, layout), and fixers for **Responsive** layout, **Optimize** (raycast targets, canvases) and **Accessibility**. Requires the Unity UI package.



UI/UX Tools page.

5.13 Optimization

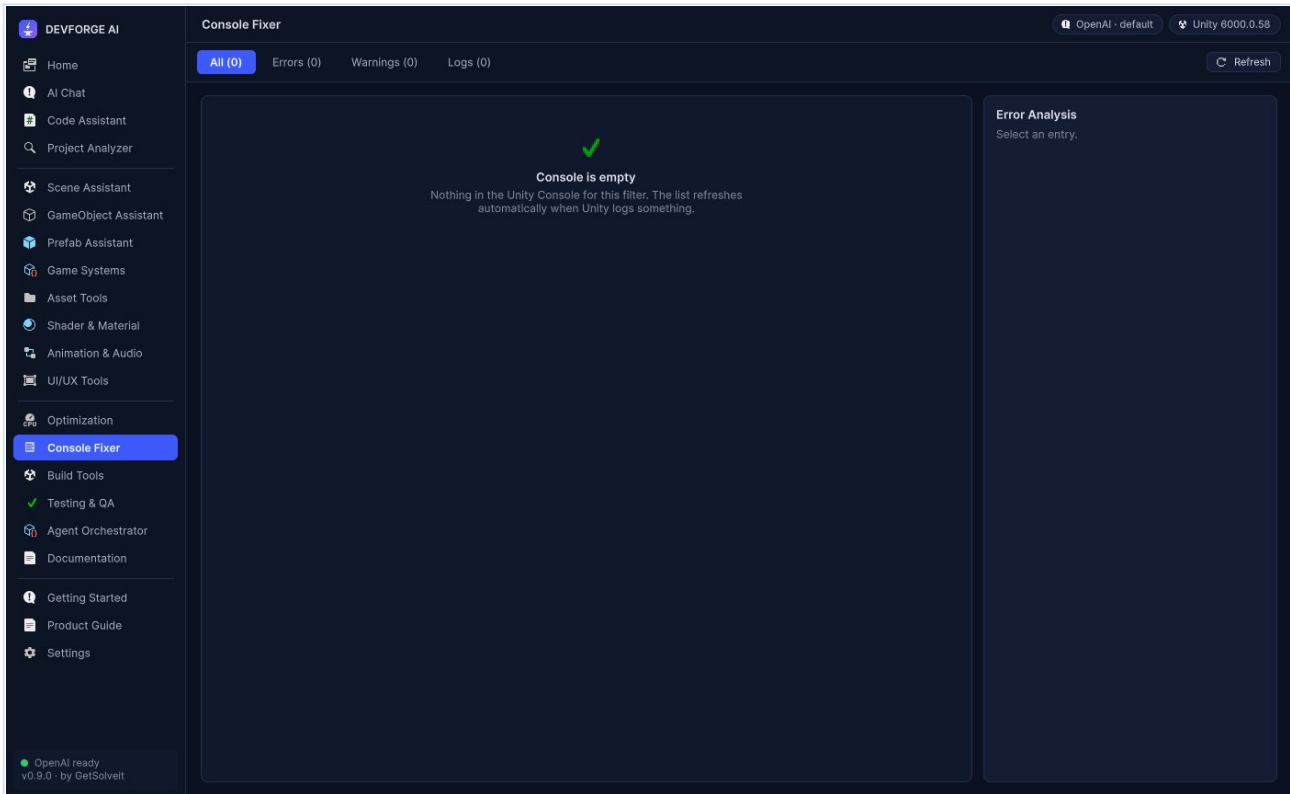
A project score plus tabs for **Performance** (live profiler stats in Play Mode, script hot-path scan, scene checks), **Mobile**, **WebGL**, **Graphics**, **Memory** and **Build Size**. Settings changes are recorded and can be rolled back.



Optimization page.

5.14 Console Fixer

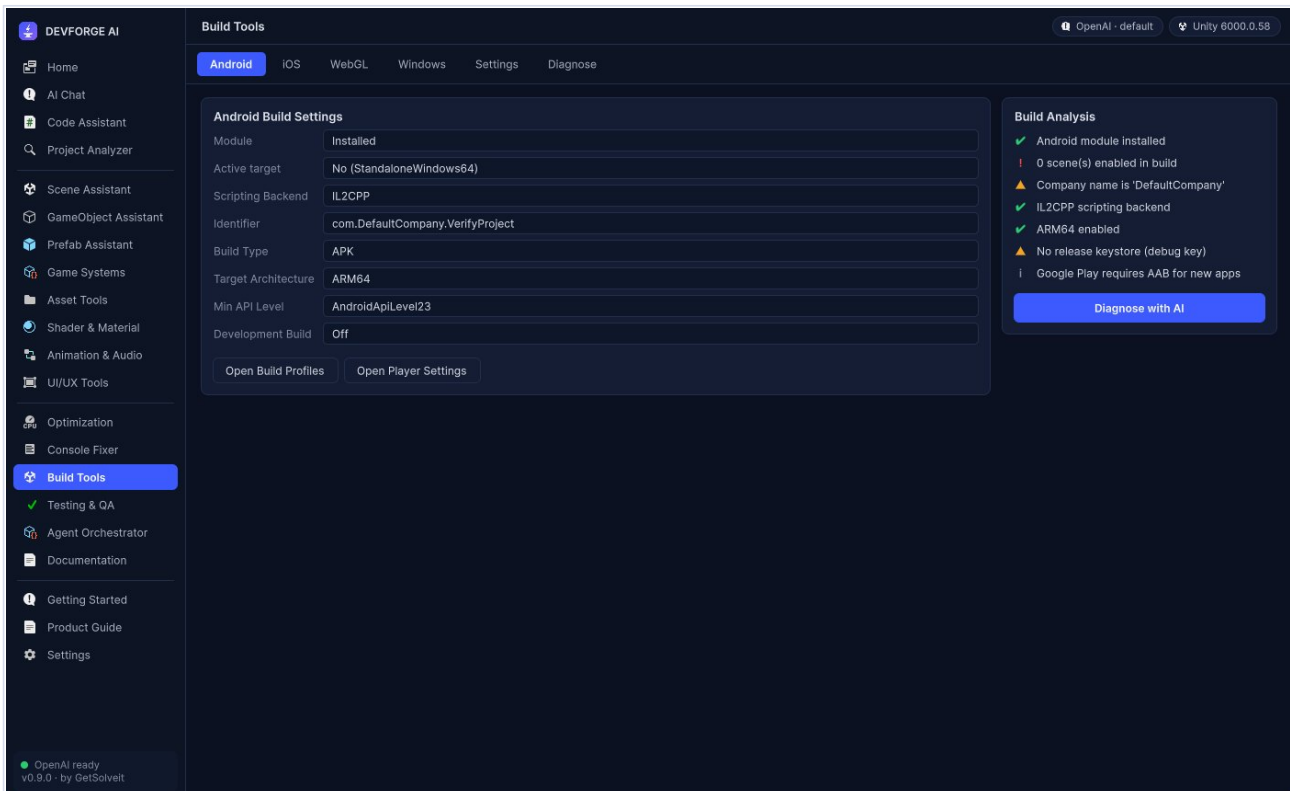
All Unity Console messages of your project (All / Errors / Warnings / Logs), independent of the Console window's filters and refreshed automatically; DevForge AI's own log messages are left out (see Settings › Diagnostics). For each entry: type, file, line, occurrences, a local cause/fix hint, **Open File**, **Explain** (AI Chat) and **Fix with Code Agent**.



Console Fixer page.

5.15 Build Tools

Per-platform settings and checks for **Android**, **iOS**, **WebGL** and **Windows**, player **Settings**, and **Diagnose** (platform fixes plus AI analysis of the last build log).

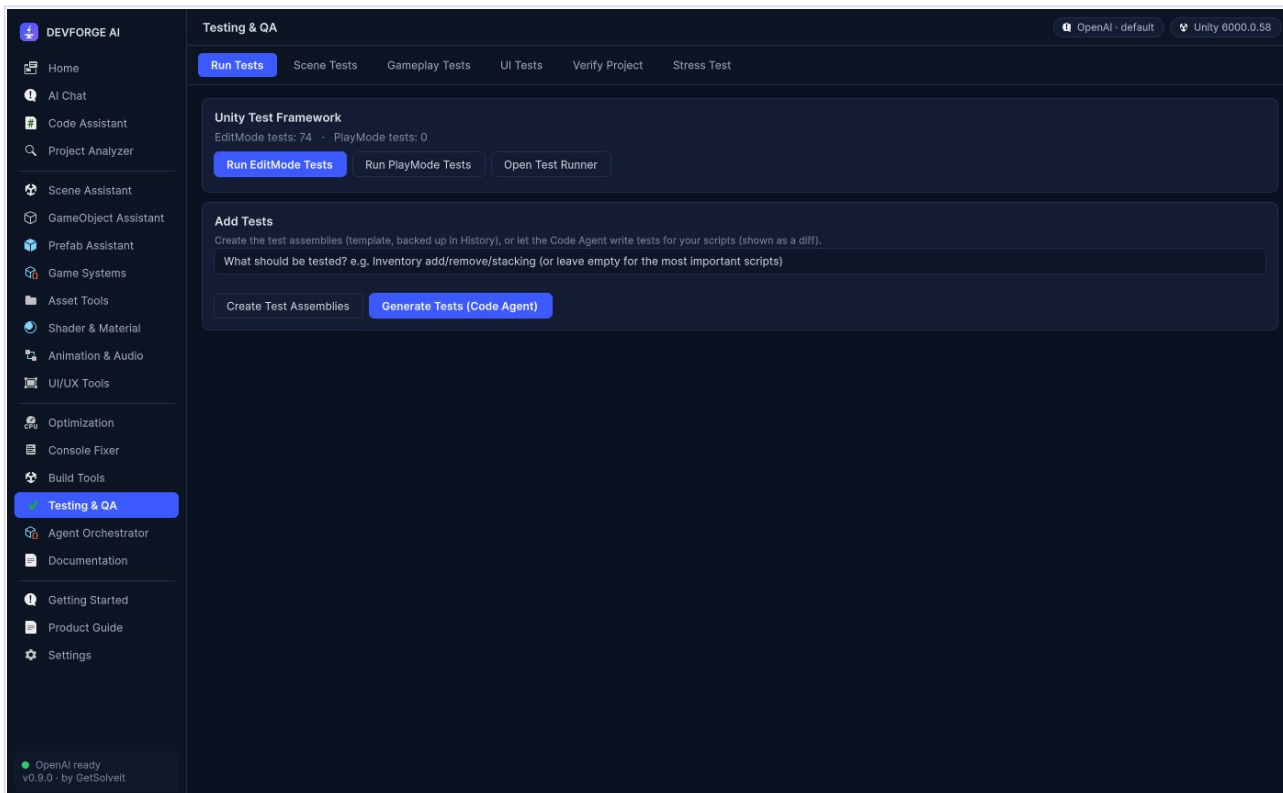


Build Tools page.



5.16 Testing & QA

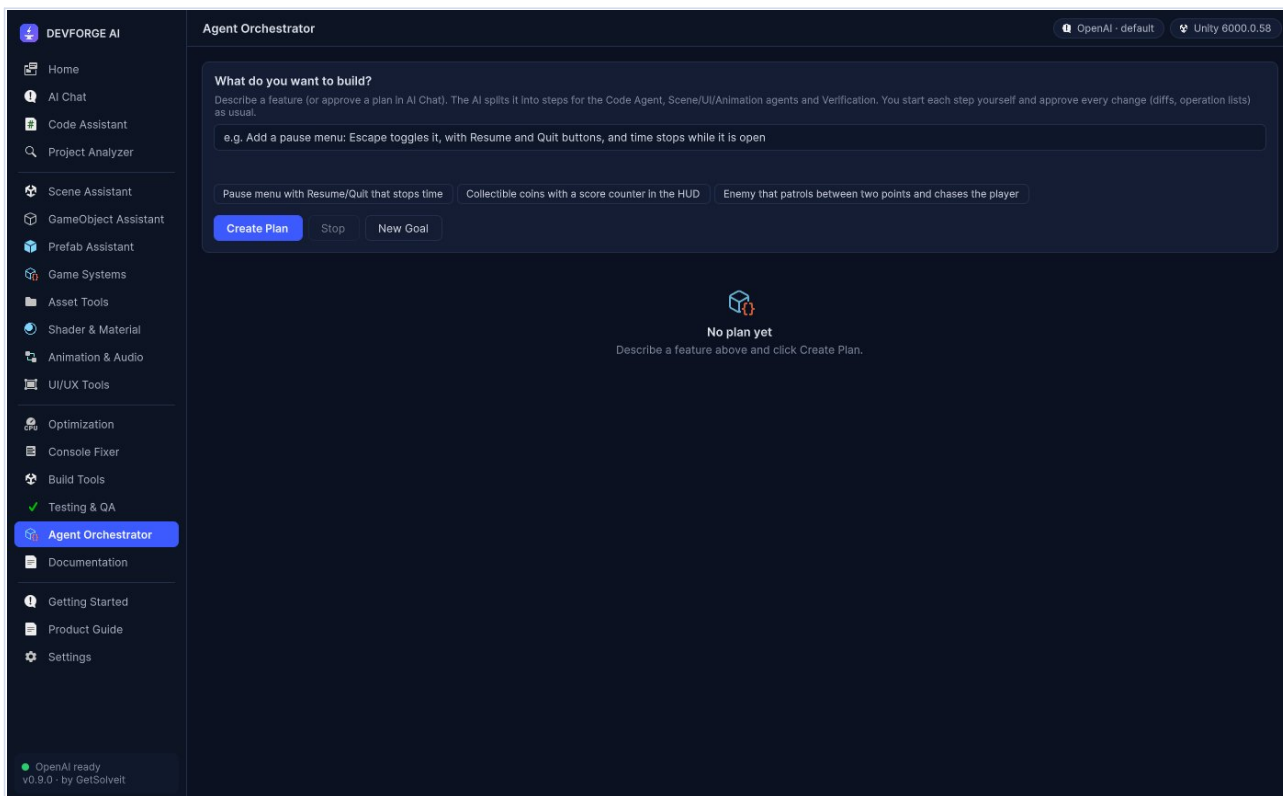
Run Tests (Unity Test Framework, EditMode/PlayMode), **Scene Tests**, **Gameplay Tests** (smoke test that enters Play Mode and watches for errors), **UI Tests** and **Verify Project** (compile, console, missing references, scene checks and tests in one report). Tests can be generated with the Code Agent. *Stress Test* is planned.



Testing & QA page.

5.17 Agent Orchestrator

Describe a feature (or approve a plan in AI Chat). The AI splits it into steps for the **code**, **scene**, **ui**, **animation**, **chat** and **verify** agents. You run each step, review its changes, and can Retry, Mark Done or Skip. When all steps are finished, a Final Report is shown and can be copied or sent to AI Chat.



Agent Orchestrator page.

5.18 Documentation

Documentation for **your** project, generated locally from your code (no AI, no usage): **Scripts** (outline of one script), **Game Systems** (scripts grouped into gameplay systems with their Inspector fields and public methods), **API Docs** (public types by namespace), **README** (project info, how to open and run it, scenes in the build, folder structure, packages and your .ai notes) and **Architecture** (assemblies and their dependencies, namespaces, which components reference which, scenes). Every document can be copied, saved to a file you choose with **Save As...** (replacing a file asks first), or sent to AI Chat with **Improve with AI**. The screenshots use a small example project.



DEVFORGE AI

Home

AI Chat

Code Assistant

Project Analyzer

Scene Assistant

GameObject Assistant

Prefab Assistant

Game Systems

Asset Tools

Shader & Material

Animation & Audio

UI/UX Tools

Optimization

Console Fixer

Build Tools

Testing & QA

Agent Orchestrator

Documentation

Getting Started

Product Guide

Settings

OpenAI ready
v0.9.0 · by GetSolveit

Documentation

ScriptsGame SystemsAPI DocsREADMEArchitecture

Game Systems

Groups your scripts into gameplay systems (player, camera, UI, economy, combat, enemies, spawning, save/load, audio...) using script and folder names, with each script's Inspector fields and public methods.
i Generated locally from your project: no AI and no usage. Your project files are not changed.

Generate

Preview

Game Systems - VerifyProject
Generated by DevForge AI on 2026-09-30 from 4 scripts (grouped by script and folder names; review and adjust).
System	Scripts
Player & Controls	1
Game Flow & Levels	1
UI & HUD	1
Score, Economy & Rewards	2
Items & Inventory	1

Player & Controls
- **PlayerController** (MonoBehaviour, 'Assets/Scripts/Player/PlayerController.cs')
 - Inspector: laneWidth, forwardSpeed
 - Public: MoveLeft(), MoveRight()

Game Flow & Levels
- **GameManager** (MonoBehaviour, 'Assets/Scripts/Gameplay/GameManager.cs')
 - Inspector: player, score
 - Public: GameOver(), Restart()

UI & HUD
- **ScoreManager** (MonoBehaviour, 'Assets/Scripts/UI/ScoreManager.cs')
 - Inspector: scoreText
 - Public: Add()

Score, Economy & Rewards
- **CoinPickup** (MonoBehaviour, 'Assets/Scripts/Gameplay/CoinPickup.cs')
 - Inspector: value, scoreManager
- **ScoreManager** (MonoBehaviour, 'Assets/Scripts/UI/ScoreManager.cs')
 - Inspector: scoreText
 - Public: Add()

Items & Inventory
- **CoinPickup** (MonoBehaviour, 'Assets/Scripts/Gameplay/CoinPickup.cs')
 - Inspector: value, scoreManager

CopySave As...Improve with AI

Documentation > Game Systems (example project).

DEVFORGE AI

Home

AI Chat

Code Assistant

Project Analyzer

Scene Assistant

GameObject Assistant

Prefab Assistant

Game Systems

Asset Tools

Shader & Material

Animation & Audio

UI/UX Tools

Optimization

Console Fixer

Build Tools

Testing & QA

Agent Orchestrator

Documentation

Getting Started

Product Guide

Settings

OpenAI ready
v0.9.0 · by GetSolveit

Documentation

ScriptsGame SystemsAPI DocsREADMEArchitecture

Architecture

How your code is organised: assemblies and their dependencies, namespaces, MonoBehaviours with the project types they reference in the Inspector, and scenes.
i Generated locally from your project: no AI and no usage. Your project files are not changed.

Generate

Preview

Architecture - VerifyProject
Generated by DevForge AI on 2026-09-30 from the project's assemblies, namespaces and scripts.

Assemblies
Assembly	Scripts	Depends on (project assemblies)
Assembly-CSharp	4	-

Namespaces
- **(global)**: 4 types (4 MonoBehaviour, 0 ScriptableObject)

Components and their dependencies
MonoBehaviours with serialized references to other project types (what is wired in the Inspector):
- **CoinPickup** → ScoreManager
- **GameManager** → PlayerController, ScoreManager

Scenes
- (none)

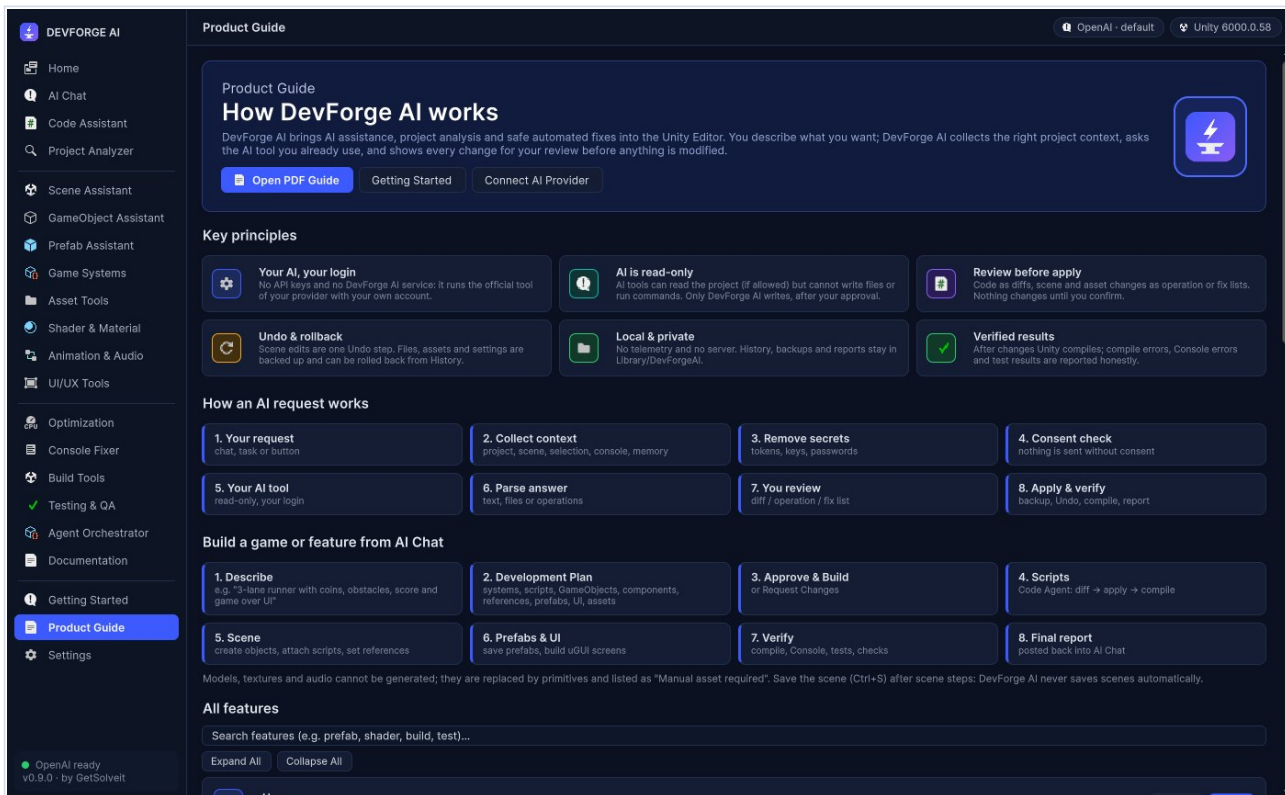
CopySave As...Improve with AI

Documentation > Architecture: assemblies, namespaces and component references (example project).



5.19 Product Guide (in the editor)

The **Product Guide** page (sidebar, or **Tools > DevForge AI > Product Guide**) explains the product inside Unity: the key principles, how an AI request works, the build-from-chat workflow, and **every feature** with *What it does*, its tabs, *How it works* step by step and its *Safety* rules. Search filters the features; **Open** jumps straight to the tool, and **Open PDF Guide** opens this document.



Product Guide page: principles, workflows and the searchable feature list.

5.20 Settings

Privacy & Data shows your consent, a table of where data goes for each provider (the active one is marked), exactly what is sent with each request (with your current choices), how your data is protected, **provider-specific settings** (the active provider is opened; for Claude Code this is where you can allow a trusted project's own hooks and MCP servers, which are ignored by default for safety) and the data stored on this computer. **About** shows the product information (Developed by GetSolveit) and all documentation, which you can read inside Unity or open as files.



DEVFORGE AI

Home

AI Chat

Code Assistant

Project Analyzer

Scene Assistant

GameObject Assistant

Prefab Assistant

Game Systems

Asset Tools

Shader & Material

Animation & Audio

UI/UX Tools

Optimization

Console Fixer

Build Tools

Testing & QA

Agent Orchestrator

Documentation

Getting Started

Product Guide

Settings

OpenAI ready
v0.9.0 · by GetSolveit

Settings

AI ProviderProject ContextPrivacy & DataDiagnosticsAbout

Your consent

When you use an AI feature, DevForge AI sends the request to the AI provider you chose in Settings > AI Provider, using your own login in the official tool: Claude Code, OpenAI Codex CLI, Google Antigravity CLI or Cursor CLI on this computer (or a local model server). It sends your message plus the project context you enabled in Settings > Project Context: project info, installed packages, the active scene hierarchy, the selected object's components and field values, Console errors and the .ai/.md memory files. If you allow file reading, the AI tool may also read files inside this project folder in its read-only mode (it cannot change files or run commands); with Claude Code, .env files, signing keys, keystores and UserSettings are blocked. Likely secrets in the collected context are redacted. The data is processed by the provider you chose (Anthropic, OpenAI, Google, Cursor or your own model server) under the terms of your account. DevForge AI has no AI service of its own and stores no API keys; chat history, task history and backups stay on this computer (Library/DevForgeAI and Unity's EditorPrefs).

I understand what is sent and agree to use the AI features ✓

Without consent nothing is sent to any AI provider. Local tools (analyzers, fixers, tests, documentation generators) always work.

Where your data goes

Only the ACTIVE provider receives data: currently OpenAI. Change it in Settings > AI Provider.

Provider	Data is sent to	Sign-in	Can the AI change files?
Claude	Anthropic, through Claude Code on this computer	Your Claude account	No: only read tools (Read, Grep, Glob)
OpenAI (active)	OpenAI, through Codex CLI on this computer	Your ChatGPT account	No: read-only sandbox
Gemini (Antigravity)	Google, through Antigravity CLI on this computer	Your Google account	No: writes and commands are denied
Cursor	Cursor, through Cursor CLI on this computer	Your Cursor account	No: ask mode, never --force
Custom (local model)	Only the server address you enter (e.g. Ollama on this computer)	None / optional token	No: it cannot read or change files

What is sent with each request

Your message or task, plus the context below (your current choices). Agents also send their task and the results of earlier steps.

✓ Project info: product name, Unity version, render pipeline, build target, asset counts, packages

✓ Project memory: your .ai/*.md notes (empty templates are not sent)

✓ Active scene: object names and component types (up to ~80 lines)

✓ Selected object: components and visible field values (password/token/secret/key fields are hidden)

✓ Console: first lines of up to 10 errors/warnings

✓ File reading: the AI tool may open project files itself (read-only)

Change In Project Context

How your data is protected

✓ No API keys: DevForge AI never asks for, stores or sends API keys. Each official tool keeps its own login.

✓ Secrets are removed from the context before sending (API keys, tokens, private keys, password= values). Best effort: keep secrets out of scenes and scripts.

Settings > Privacy & Data.

DEVFORGE AI

Home

AI Chat

Code Assistant

Project Analyzer

Scene Assistant

GameObject Assistant

Prefab Assistant

Game Systems

Asset Tools

Shader & Material

Animation & Audio

UI/UX Tools

Optimization

Console Fixer

Build Tools

Testing & QA

Agent Orchestrator

Documentation

Getting Started

Product Guide

Settings

OpenAI ready
v0.9.0 · by GetSolveit

Settings

AI ProviderProject ContextPrivacy & DataDiagnosticsAbout

DevForge AI

AI-powered game development toolkit for the Unity Editor

Version 0.9.0 · Developed by GetSolveit

Product

Product: DevForge AI

Version: 0.9.0

Developed by: GetSolveit

Unity: 6000.0 (Unity 6) or newer · this project: 6000.0.58f2

AI tools: One of: Claude Code, OpenAI Codex CLI, Google Antigravity CLI, Cursor CLI, or a local model server. You use your own account; no API keys.

Active provider: OpenAI (Ready)

Copyright: © 2026 GetSolveit. All rights reserved.

Documentation

Read the documentation right here, or open the illustrated PDF guide.

Open PDF Guide

Product Guide (every feature)

Show Documentation Folder

User Guide

Setup, how changes are applied, every page, tips and troubleshooting.

AI Providers

Install and log in: Claude Code, Codex CLI, Antigravity CLI, Cursor CLI, local servers.

Privacy

What is sent, where it goes and what stays on this computer.

Security

Read-only AI, protected files, change safety and remaining risks.

Compatibility

Unity versions, operating systems, pipelines and packages.

Known Limitations

What does not work yet and what to watch out for.

Changelog

What changed in each version.

Settings > About: product information and documentation reader.

Tab	Contents
AI Provider	Providers, login, model, Test Connection, active provider (see 3.1).
Project Context	What is attached to AI requests; create missing .ai memory files.

DevForge AI · Developed by GetSolveit

Page 27



Tab	Contents
Privacy & Data	Consent, where data goes per provider, what is sent, protections, provider-specific settings, local data and clean-up.
Diagnostics	Versions, provider status and environment; Copy to Clipboard for support requests.
About	Product information (Developed by GetSolveit) and all documentation: read inside Unity, open the files or the PDF guide.

6. Safety, Privacy & Data

6.1 What leaves your computer

Only when you use an AI feature **and** have accepted the data notice, DevForge AI sends a request to the active provider through its official tool. The request contains your message or task, the context you enabled (project info, .ai memory, scene hierarchy, selected object, Console errors) and, for agents, their task and results of earlier steps. The AI tool may read project files itself in its read-only mode if file reading is enabled. Processing is governed by the terms of **your** account with that provider.

6.2 What stays on your computer

Data	Location
Chat history, agent and orchestrator state	Library/DevForgeAI/*.json
Task history and backups	Library/DevForgeAI/history/
Test and smoke test results	Library/DevForgeAI/qa/
Settings, consent, recent activity	Unity EditorPrefs (per user)
Temporary prompt files (Antigravity, Cursor)	Library/DevForgeAI/tmp/ (deleted after each request)

6.3 Protections

- Consent gate: no project data is sent before you accept the data notice.
- Secret scrubber: common token formats and sensitive key/value pairs are removed; fields named like password/token/secret/key are never included.
- Claude Code cannot read .env files, private keys, keystores, provisioning profiles or UserSettings/.
- No API keys are entered in or stored by DevForge AI; tool paths and model names are validated before use.
- Writes only inside Assets/, never inside the DevForge AI folder, never without review; backups for every change.
- Requests are cancelled on script reload and Editor quit; Stop ends the tool's whole process tree.



7. Troubleshooting

Message / symptom	What to do
"... not found"	Install the tool, restart Unity, or use Browse in Settings > AI Provider.
"... is not logged in" / 401 Unauthorized	Click Log in and finish the sign-in in the browser. Codex: check <i>codex login status</i> . Signing in on a website or in an IDE extension does not log in the CLI.
IneligibleTierError (Gemini)	Gemini CLI is retired for individual accounts. Install the Antigravity CLI (agy); DevForge AI switches automatically.
'chcp' is not recognized	Your Windows PATH lacks C:\Windows\System32. DevForge AI adds it for the tools it starts; consider repairing your PATH.
"Please review and accept the data notice"	Getting Started step 2 or Settings > Privacy & Data.
Usage limit reached	Your provider plan's limit; wait or change plan/provider.
Answers ignore project rules	Fill in .ai/*.md and check the Project Context toggles.
A new script cannot be added as a component	Run the code step first and wait until Unity has compiled, then run the scene step.
Log in opens an Antigravity window with a > prompt	That is the Antigravity CLI and you are already signed in; close it and click Test Connection. (Newer builds show "already logged in" instead.)
Gemini opens the Antigravity IDE or never answers	The path points to Antigravity.exe (the IDE). Clear the path so agy.exe is detected.
"Antigravity tried an action that is blocked..."	The AI tried a terminal command twice. Ask more specifically, or switch provider for that question.
The PDF guide does not open	Use Settings > About > Show Documentation Folder and open DevForgeAI_Documentation.pdf with any PDF viewer; every document can also be read inside Unity (About > Read).
Anything else	Settings > Diagnostics > Copy to Clipboard and include it in your support request.

8. Limitations & Roadmap

8.1 Known limitations

- Codex, Antigravity and Cursor answer when complete (no live streaming); earlier chat turns are sent as a short transcript.
- Cursor CLI was verified only against a test double; Claude Code, Codex CLI and Antigravity CLI were tested with real accounts.
- macOS and Linux are supported by the code but not yet tested.
- AI output can be wrong: always review diffs and operation lists, use version control, and run Verify Project.
- 3D models, textures and audio cannot be generated; they are reported as *Manual asset required*.



8.2 Planned features

- Navigation Assistant (Scene Assistant)
- Animation Retargeting
- Stress Test (Testing & QA)

8.3 Tips

- Keep .ai/*.md short and current; they are sent with every request.
- Commit to version control before large Code Agent or Orchestrator runs; backups are a safety net, not a replacement.
- Save the scene (Ctrl+S) after scene steps (DevForge AI never saves scenes automatically), and use Verify Project before builds.